



PUSSYCAT

Security Review



APRIL, 2026

www.chaindefenders.xyz
<https://x.com/DefendersAudits>

Lead Auditors



PeterSR



0x539.eth

Intern Auditors



priyanshukumar397

Table of Contents

1. Protocol Summary
2. Disclaimer
3. Risk Classification
4. Audit Details
 - Commit Hash
 - Scope
 - Roles
5. Executive Summary
 - Issues Found
 - Issues Statuses
 - Security Grade
6. Findings
 - High
 - Medium

- Low
- Informational

Protocol Summary

Pussycat is a Solana blockchain smart contract (program) built with Anchor that implements an on-chain cat breeding and gaming system. The contract features:

Game mechanics: Players can join, place and upgrade farms, and claim rewards

Breeding system: Request-based breeding with VRF (Verifiable Random Function) randomness for outcome resolution

Marketplace: Dual marketplace supporting both direct sales and siring services

Admin controls: Initialization, configuration management, pause mechanisms, and operator oversight

Utilities: On-chain reward estimation and related helper functions

Disclaimer

The Chain Defenders team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the implementation of the contracts.

Risk Classification

Impact/Likelihood	High	Medium	Low
High	High	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Audit Details

Commit Hash

173f16f76987714a32f6b53b973c5c004ced99d0

Scope

Id	Files in scope
1	programs/game/src/constants.rs
2	programs/game/src/errors.rs
3	programs/game/src/events.rs
4	programs/game/src/lib.rs
5	programs/game/src/state.rs
6	programs/game/src/instructions/mod.rs
7	programs/game/src/instructions/admin/config.rs
8	programs/game/src/instructions/admin/emergency.rs
9	programs/game/src/instructions/admin/initialize.rs
10	programs/game/src/instructions/admin/mod.rs
11	programs/game/src/instructions/admin/operator.rs
12	programs/game/src/instructions/admin/pause.rs
13	programs/game/src/instructions/breeding/mod.rs
14	programs/game/src/instructions/breeding/request.rs
15	programs/game/src/instructions/breeding/resolve.rs
16	programs/game/src/instructions/breeding/vrf.rs
17	programs/game/src/instructions/game/farm.rs
18	programs/game/src/instructions/game/join.rs
19	programs/game/src/instructions/game/mod.rs
20	programs/game/src/instructions/game/rewards.rs

Id	Files in scope
21	programs/game/src/instructions/marketplace/listing_validation.rs
22	programs/game/src/instructions/marketplace/mod.rs
23	programs/game/src/instructions/marketplace/sale.rs
24	programs/game/src/instructions/marketplace/siring.rs
25	programs/game/src/utls/emission.rs
26	programs/game/src/utls/marketplace_fee.rs
27	programs/game/src/utls/mod.rs
28	programs/game/src/utls/signature.rs
29	programs/game/src/utls/token.rs
30	programs/game/src/utls/vrf_helpers.rs

Roles

Id	Roles
1	Admin
2	Operator
3	Signature Authority
4	Treasury
5	User

Executive Summary

Issues Found

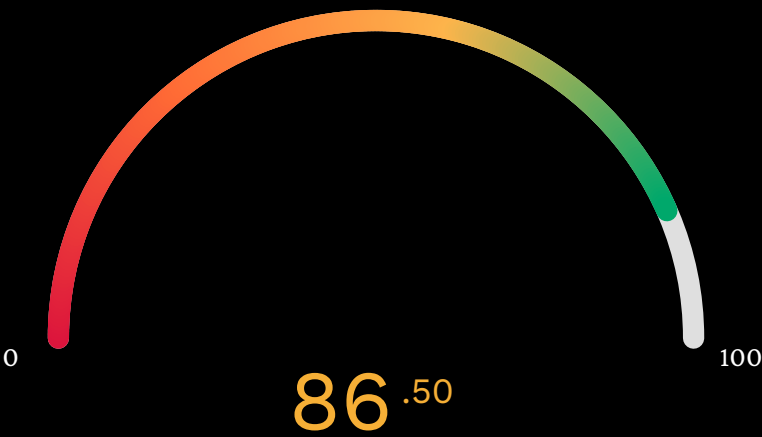
Severity	Count	Description
High	1	Critical vulnerabilities
Medium	4	Significant risks
Low	6	Minor issues with low impact
Informational	5	Best practices or suggestions
Gas	0	Optimization opportunities

Issues Statuses

Id	Title	Status
H-01	'join_game' Increments Global Grow Power Without Pre-Harvest — Retroactively Dilutes Existing Players' Unclaimed Rewards	Fixed
M-01	Emission Table Retroactively Updated For Already-Elapsed Unclaimed Days	Fixed
M-02	Fixed-Seed PDAs In 'initialize_system' Are Susceptible To Deployment Griefing Via Front-Running	Fixed
M-03	'attempt_id' Collision With Existing Cat PDA Bricks 'STATUS_RANDOMNESS_READY' Attempt	Fixed
M-04	DoS via PDA Pre-funding	Fixed
L-01	Starter Cat GP Is Uncapped On-chain Which Can Lead To Astronomically Powerful Farming Cats	Fixed
L-02	'admin_close_attempt' Does Not Restore Parent Cat State — 'breeding_count' And 'last_bred_at' Permanently Inflated	Fixed
L-03	'SiringUsed' Event 'matron_owner' Field Always Emits the Buyer's Key	Acknowledged
L-04	Signature Authority Rotation Immediately Invalidates All In-Flight Signed Messages	Acknowledged

Id	Title	Status
L-05	'cancel_siring_listing' Executable After SOL And CAT Already Paid By Buyer	Acknowledged
L-06	'join_fee' Not Committed In SA Signature — Admin Can Front-Run Join To Charge A Higher Fee	Acknowledged
I-01	Immutable Placeholder 'CAT_TOKEN_URI' Permanently Bricks Token Metadata Display On Deployment	Acknowledged
I-02	Admin-Configurable Economic Parameters Lack On-Chain Bounds	Acknowledged
I-03	No On-Chain Anchor Events for 12 Admin Functions	Fixed
I-04	'cancel_sale_listing' And 'cancel_siring_listing' Missing Explicit 'cat.owner == seller' Constraint	Fixed
I-05	'placed_cat_ids' Array Uses '0' As Empty Sentinel — Collides With Valid Cat ID 0	Fixed

Security Grade



Based on the audit findings and code quality, the overall protocol security grade is **86.50 / 100.00**. This reflects a good security posture with room for improvement.

Findings

High

High 01 `join_game` Increments Global Grow Power Without Pre-Harvest — Retroactively Dilutes Existing Players' Unclaimed Rewards

Location

`join.rs:182-203`

Description

When a new player joins the game, `handle_join_game` immediately adds the new player's combined cat grow power (`gp_1 + gp_2`) to `global_stats.total_grow_power` without first harvesting pending rewards for any existing player. The reward formula at claim time uses the current live value of `global_stats.total_grow_power` as the denominator for the entire elapsed period since the claiming player's last harvest. This means that for any player who joined before the new entrant and has not yet claimed, their share of every reward-bearing second since their last harvest is recalculated against the inflated denominator that includes the new player's GP — even though the new player was not present for any of that time.

```
1 // join.rs:199-203 — GlobalStats updated with no prior harvest for existing players
2 let global_stats = &mut ctx.accounts.global_stats;
3 global_stats.total_grow_power = global_stats
4     .total_grow_power
5     .checked_add(total_gp)
6     .ok_or(GameError::GrowPowerArithmetic)?;
```

The impact of the given issue is:

Existing players who harvest infrequently suffer the largest dilution. For a single existing player with GP = 1,000 who has farmed for 10 days alone, a new player joining with GP = 1,000 immediately halves the existing player's retroactive reward for the entire 10-day period — even though the new entrant contributed no grow power during those days. At day-30 emission (~126.5 million CAT/day), a player with 50% of the effective pool can lose approximately 50% of a day's allocation per day of unclaimed rewards. The effect is bounded by the `MAX_HARVEST_DAYS` cap (51 days) and attenuates as more players are in the pool.

Recommendation

Either (a) require that `join_game` is preceded by a harvesting step for all active players (impractical on-chain), or (b) adopt a time-weighted shares model (e.g., a global reward-per-GP accumulator updated atomically whenever the total GP changes) so that each second of emission is credited at the correct denominator for that second. Option (b) is the standard approach used by Synthetix-style staking contracts and prevents all retroactive dilution regardless of join/leave timing.

Status

Fixed

Medium

Mid 01 Emission Table Retroactively Updated For Already-Elapsed Unclaimed Days

Location

config.rs:35-53

emission.rs:7-19

emission.rs:38-77

Description

`handle_update_emission_table` replaces the entire 51-element emission table in a single call with no guard on already-elapsed day indices:

```
1 // config.rs:35-53
2 pub fn handle_update_emission_table(
3     ctx: Context<UpdateEmissionTable>,
4     emission_table: [u64; EMISSION_TABLE_SIZE],
5     emission_terminal: u64,
6 ) -> Result<()> {
7     require!(
8         emission_table.iter().any(|v| *v > 0) || emission_terminal > 0,
9         GameError::InvalidParameter
10    );
11    let config = &mut ctx.accounts.game_config;
12    config.emission_table = emission_table; // full replacement, no day-index guard
```

```

13     config.emission_terminal = emission_terminal;
14     ...
15 }

```

The reward calculation function reads the live config at claim time, not a per-player snapshot:

```

1 // emission.rs:7-16
2 fn emission_for_day(day: u32, config: &GameConfig) -> u64 {
3     if day ≤ emission_table_size {
4         let idx = usize::try_from(day - 1).expect("...");
5         config.emission_table[idx] // reads live config, no snapshot
6     } else {
7         config.emission_terminal
8     }
9 }

```

The combination means: if a player last harvested at the start of day 5 and the admin updates the emission table on day 6 to reduce day-5 rates, the player's `claim_rewards` call on day 6 calculates their day-5 rewards at the new (lower) rate. Rewards for the entire elapsed-but-unclaimed period since `last_harvest_time` are computed using the live table at claim time, not the table in effect when those days actually elapsed.

This creates a structural asymmetry between players who claimed before an emission table update and players who claim after. The admin can both reduce rates (penalizing late-claiming players) and increase rates (creating incentives to delay claiming).

The impact of this issue is the following:

- Players who harvest infrequently (weekly or longer) are most exposed — up to 6+ day-buckets per harvest cycle can be retroactively repriced.
- At peak day-7 emission (~411,651 CAT/day global), reducing a single day to 0 after it has elapsed could deny thousands of CAT across all active players who have not yet claimed.
- The effect is particularly severe for players with large deferred windows — see M-10 for the compound scenario where the 51-day claim cap creates an inescapable multi-transaction window exposed to exactly this repricing.

Recommendation

Add a current-day guard so that only future day indices can be modified. The current season day can be computed on-chain at update time:

```

1 pub fn handle_update_emission_table(
2     ctx: Context<UpdateEmissionTable>,
3     emission_table: [u64; EMISSION_TABLE_SIZE],
4     emission_terminal: u64,

```

```

5  ) -> Result<> {
6      let config = &ctx.accounts.game_config;
7      let now = Clock::get()?.unix_timestamp;
8      let current_day = season_day(now, config) as usize;
9
10     // Allow changes only for days that have not yet started
11     // For already-elapsed entries, require the new value matches the existing value
12     for i in 0..current_day.min(EMISSION_TABLE_SIZE) {
13         require!(
14             emission_table[i] == config.emission_table[i],
15             GameError::InvalidParameter // cannot reprice elapsed days
16         );
17     }
18     ...
19 }

```

Alternatively, store a checkpoint mapping `day_index -> rate_at_start_of_day` per player (higher storage cost but provides per-player protection regardless of admin behavior).

Status

Fixed

Mid 02 Fixed-Seed PDAs In `initialize_system` Are Susceptible To Deployment Griefing Via Front-Running

Location

`initialize.rs:24-31`

`initialize.rs:33-39`

Description

Three PDAs created during `initialize_system` — `global_stats`, `cat_mint`, and `mint_authority` — are derived from fixed seeds with no caller-supplied component:

```

1 // global_stats: seeds = [b"global_stats"]
2 // cat_mint:      seeds = [b"cat_mint"]
3 // mint_authority: seeds = [b"mint_authority"]

```

Because the PDA addresses are fully deterministic and publicly knowable before deployment, a griever can compute them off-chain and pre-create them (by sending lamports or initializing them with arbitrary data) before the deployer calls `initialize_system`. Anchor's `init` constraint requires the account to not yet exist — if the account already exists at the expected address, the instruction fails with `AccountAlreadyInitialized`.

This is in contrast to `game_config`, which includes a caller-supplied `config_seed: [u8; 32]` in its derivation. If a griever pre-creates `game_config` at a given seed, the deployer can simply choose a different seed. No such escape hatch exists for the three fixed-seed PDAs.

The impact of the given issue is:

A griever can permanently block this specific program deployment at zero cost (only rent lamports needed to create the accounts). The program cannot be re-initialized without a fresh deploy at a new program ID. This is a one-time deployment-window attack with no ongoing user fund risk — all live user funds are safe once the program is successfully deployed. The practical risk is low for a single-deploy program but meaningful if the team ever needs to redeploy (post-upgrade, emergency re-initialization), as any known program ID becomes permanently grievable.

Recommendation

Include the admin's public key (or a deployer-supplied nonce) as a seed component for `global_stats`, `cat_mint`, and `mint_authority`, so that their addresses are not knowable until the deployer broadcasts the transaction. Alternatively, submit `initialize_system` as the first transaction from a fresh wallet via a private RPC relay (e.g., Jito bundle) to close the front-running window.

Status

Fixed

Mid 03 attempt_id Collision With Existing Cat PDA Bricks STATUS_RANDOMNESS_READY Attempt

Location

resolve.rs:52-55

request.rs:50-56

Description

The child `Cat` PDA created by `resolve_breeding` is seeded with `[CAT_SEED, 8breeding_attempt.attempt_id.to_le_bytes()]` (`resolve.rs:52`), making the new cat's on-chain ID equal to `attempt_id`. The `attempt_id` is a `u32` supplied freely by the player at `request_breeding` time with no uniqueness check against existing cat IDs. If a player submits `attempt_id = X` where a `Cat` PDA for ID `X` already exists, the `BreedingAttempt` PDA is successfully created (seeded with `[BREEDING_SEED, X]` — different prefix, no collision) and the VRF request is dispatched normally. When the operator later calls `resolve_breeding`, Anchor's `init` constraint on `new_cat` fails with “account already in use” because `[CAT_SEED, X]` is already occupied.

```
1 // resolve.rs:48-55 — init fails if cat_id X already exists
2 #[account(
3     init,
4     payer = operator,
5     space = Cat::SIZE,
6     seeds = [CAT_SEED, 8breeding_attempt.attempt_id.to_le_bytes()],
7     bump,
8 )]
9 pub new_cat: Box<Account<'info, Cat>>,
```

The attempt is now permanently stuck in `STATUS_RANDOMNESS_READY`. `admin_close_attempt` cannot recover it because that instruction requires `STATUS_AWAITING_RANDOMNESS` (`emergency.rs:136`). No other instruction closes a `STATUS_RANDOMNESS_READY` attempt. The `BreedingAttempt` PDA account is frozen on-chain with no reclaim path for the operator rent.

The impact of the given issue is:

- The player loses their breeding cost (CAT tokens burned at request time) — self-inflicted but permanent.
- Both parent cats have their `breeding_count` permanently incremented and `last_bred_at` frozen, consuming a cooldown slot and extending perceived cooldown with no offspring produced.
- The `BreedingAttempt` PDA rent is locked on-chain indefinitely (no close path for this status).
- An operator spam scenario: if many players deliberately trigger this, the operator accumulates orphaned `STATUS_RANDOMNESS_READY` accounts with no cleanup mechanism. Combined with M-02 (no recovery for this status), this creates a class of permanently irrecoverable state.

Recommendation

Two complementary fixes:

1. At request time: validate that `[CAT_SEED, &attempt_id.to_le_bytes()]` does not already exist, or require `attempt_id` to come from a server-issued monotonic sequence included in the SA signature (the SA already signs `attempt_id` — it should enforce global uniqueness).
2. At admin level: extend `admin_close_attempt` to also handle `STATUS_RANDOMNESS_READY` attempts, so stuck attempts of any status can be recovered and rent returned to the player.

Status

Fixed

Mid 04 DoS via PDA Pre-funding

Location

`join.rs`

Description

The `handle_join_game` function is vulnerable to a DoS (Denial of Service) via PDA Pre-funding.

In Solana, an account can be created in two ways: by the System Program (becoming a Data account) or by simply receiving lamports (remaining a System Program owned account with zero data).

The `JoinGame` struct uses the Anchor `init` constraint for several accounts: `player_state`, `cat_1`, `cat_commitment_1`, `cat_2`, and `cat_commitment_2`. If any of these Program Derived Addresses (PDAs) already exist and have a non-zero balance—even if they have no data and are owned by the System Program—the Anchor `init` macro will fail with a `0x0` or `AlreadyInUse` error.

Because the seeds for `cat_1`, `cat_2`, and their commitments are based on `cat_id`, an attacker can monitor the transaction mempool or guess upcoming Sequential IDs and send a small amount of lamports (e.g., 1 lamport) to those PDA addresses. Once pre-funded, the legitimate `handle_join_game` transaction will always fail when it attempts to initialize those accounts.

Proof of Concept

1. Observation: An attacker identifies that `cat_id_1` and `cat_id_2` are passed as arguments and used as seeds: `seeds = [CAT_SEED, &cat_id_1.to_le_bytes()]`.
2. Targeting: The attacker calculates the PDA for the next expected Cat IDs: `rust let (pda, _bump) = Pubkey::find_program_address(&[b"cat", &next_id.to_le_bytes()], program_id);`

3. Attack: The attacker sends 0.000001 SOL to that `pda` address.
4. Failure: A new player attempts to join the game using `next_id`. The Solana runtime sees that the account `pda` already exists (it has lamports). Anchor's `init` logic attempts to execute a `create_account` instruction. The System Program rejects this because the account already exists and has a balance, resulting in a failed transaction for the user.
5. Persistence: The attacker can automate this for all future IDs, effectively preventing any new players from joining the game indefinitely.

Recommendation

To fix this, you should move away from the standard `init` constraint for these accounts and implement a “Manual Initialization” or “Reclaim” pattern. This involves:

1. Changing the account types in the struct to `AccountInfo`.
2. In the handler, check if the account has lamports but is still owned by the System Program.
3. If it is pre-funded, either:
 - Reclaim: Transfer the existing lamports back to the `player` or include them in the rent-exemption calculation.
 - Zero-Copy/Manual Init: Use `invoke_signed` to `create_account` only if the data length is 0, or use `CpiContext` to allocate space and assign the program as the owner regardless of the existing balance.

The most robust way is to use a manual initialization helper that detects if an account is “funded but empty” and proceeds with `allocate` and `assign` instead of a strict `create_account` call which fails on existing balances.

Status

Fixed

Low

Low 01 Starter Cat GP Is Uncapped On-chain Which Can Lead To Astronomically Powerful Farming Cats

Location

`join.rs`

Description

`resolve_breeding` in `resolve.rs` explicitly enforces `require!(gp ≤ config.max_fancy_gp, GameError::GpExceedsMax)` before writing GP to a child cat. `join_game` contains no equivalent guard. The `gp_1` and `gp_2` arguments are SA-signed, but the on-chain instruction accepts any value the SA embeds in the message. The backend computes GP from off-chain gene data in NeonDB (per the developer comment in `resolve.rs`: “GP is computed off-chain from genes”). If the backend has a calculation bug, a rounding overflow, or a type-coercion error that produces a GP value above `max_fancy_gp`, the inflated GP is written to the Cat PDA and immediately added to `global_stats.total_grow_power` with no ceiling. The SA is a single hot key signing every player join automatically - a backend bug here affects every join until patched, unlike operator keys which are used per-breeding. Two starter cats with `gp = u32::MAX` (4,294,967,295 each) produce `total_grow_power = 8.59 × 109`, which is ~28,600x the intended maximum of `2 × max_fancy_gp = 600,000`. All existing and future legitimate players earn near-zero rewards because `user_gp / total_gp` approaches zero for every claim in `compute_pending_rewards`.

Proof of Concept

Trace through the code with `gp_1 = u32::MAX`, `gp_2 = u32::MAX`:

`join.rs` handler - no guard:

```
1 // After SA signature passes (gp values are in the signed message):
2 cat_1.gp = gp_1; // = 4_294_967_295 - no check
3 cat_2.gp = gp_2; // = 4_294_967_295 - no check
4 let total_gp = u64::from(gp_1) + u64::from(gp_2); // = 8_589_934_590
5 player_state.total_grow_power = total_gp;
6 global_stats.total_grow_power = global_stats
7     .total_grow_power
8     .checked_add(total_gp) // adds 8.59B to global denominator
9     .ok_or(GameError::GrowPowerArithmetic)?;
```

`resolve.rs` - has the guard (missing from `join.rs`):

```
1 require!(gp ≤ config.max_fancy_gp, GameError::GpExceedsMax); // 300_000 cap
```

`emission.rs` - reward formula at claim time:

```
1 total_reward += daily_emission * elapsed * u128::from(user_gp)
2 / (u128::from(total_gp) * u128::try_from(SECONDS_PER_DAY) ... );
```

Day-7 peak emission = 411_651_459_000_000. Legitimate player with `user_gp = 100`:

```

1 reward = 411_651_459_000_000 x 86_400 x 100 / (8_589_934_590 x 86_400)
2         = 41_165_145_900_000_000_000 / 741_880_988_160_000
3         = 55 base units (0.000055 CAT per day)

```

Without the attack, same player would earn: $411_651_459_000_000 \times 86_400 \times 100 / (600_000 \times 86_400) = 68_608_576_500$ base units (~68,608 CAT per day).

The attacker's two cats, with `user_gp = 8_589_934_590`:

```

1 reward = 411_651_459_000_000 x 86_400 x 8_589_934_590 / (8_590_534_590 x 86_400)
2         = 411_651_459_000_000 x (8_589_934_590 / 8_590_534_590)
3         = 411_620_743_000_000 base units (~411,620 CAT per day, = 99.99% of all emissions
)

```

Recommendation

Add the GP cap check in `handle_join_game` immediately after signature verification, mirroring `handle_resolve_breeding`:

```

1 require!(gp_1 ≤ config.max_fancy_gp, GameError::GpExceedsMax);
2 require!(gp_2 ≤ config.max_fancy_gp, GameError::GpExceedsMax);

```

Status

Fixed

Low 02 admin_close_attempt Does Not Restore Parent Cat State — breeding_count And last_bred_at Permanently Inflated

Location

emergency.rs:115-151

Description

`handle_admin_close_attempt` is designed to recover breeding attempts stuck in `STATUS_AWAITING_RANDOMNESS` (e.g., when VRF delivery is delayed or fails). However, the instruction's account struct does not include the parent cat accounts (`matron` and `sire`), so it cannot restore the state that was set on those cats when the breeding request was created.

At request time, `handle_request_breeding` sets both `parent.status = STATUS_AWAITING_RANDOMNESS`, increments `parent.breeding_count`, and records `parent.last_bred_at = now`. When the admin closes a stuck attempt via `admin_close_attempt`, none of these fields are restored: the parent cats retain their in-breeding status, inflated `breeding_count`, and frozen `last_bred_at` — even though no child was produced.

```

1 // emergency.rs:115-151 — parent Cat accounts absent from struct
2 pub struct AdminCloseAttempt<'info> {
3     pub admin: Signer<'info>,
4     pub breeding_attempt: Account<'info, BreedingAttempt>,
5     // No matron_cat or sire_cat accounts here
6 }
```

The consequence for `breeding_count` is permanent: the counter is never decremented, consuming a cooldown slot for a breeding cycle that produced no offspring.

The impact of the given issue is:

- Parent cats retain `STATUS_AWAITING_RANDOMNESS` after their attempt is emergency-closed, blocking them from further breeding or farming until the status is manually corrected via a separate instruction.
- `breeding_count` is permanently incremented for each stuck attempt, accelerating cooldown escalation.
- `last_bred_at` remains frozen at request time, further extending the perceived cooldown window.

Recommendation

Add the parent cat accounts to the `AdminCloseAttempt` account struct and restore their state when closing a stuck attempt:

```

1 pub struct AdminCloseAttempt<'info> {
2     pub admin: Signer<'info>,
3     pub breeding_attempt: Account<'info, BreedingAttempt>,
4     #[account(mut, seeds = [CAT_SEED, 8breeding_attempt.matron_id.to_le_bytes()], bump)]
5     pub matron_cat: Account<'info, Cat>,
6     #[account(mut, seeds = [CAT_SEED, 8breeding_attempt.sire_id.to_le_bytes()], bump)]
7     pub sire_cat: Account<'info, Cat>,
8 }
9
10 // In handler body:
11 ctx.accounts.matron_cat.status = Cat::STATUS_IDLE;
12 ctx.accounts.matron_cat.breeding_count = ctx.accounts.matron_cat.breeding_count.
    saturating_sub(1);
13 ctx.accounts.sire_cat.status = Cat::STATUS_IDLE;
```

```
14 ctx.accounts.sire_cat.breeding_count = ctx.accounts.sire_cat.breeding_count.  
    saturating_sub(1);
```

Status

Fixed

Low 03 SiringUsed Event `matron_owner` Field Always Emits the Buyer's Key

Location

`siring.rs:488-494`

Description

The `SiringUsed` Anchor event includes a `matron_owner` field that, by its name, should identify the owner of the matron cat (the cat being bred as the female). The on-chain code unconditionally sets this field to `ctx.accounts.buyer.key()` regardless of who actually owns the matron. In the typical case the buyer is the matron owner, so the value appears correct. However, when the buyer is acting on behalf of a different matron owner (a valid pattern in CryptoKitties-style siring marketplaces), the emitted `matron_owner` is wrong.

```
1 // siring.rs:488-494  
2 emit!(SiringUsed {  
3     buyer: ctx.accounts.buyer.key(),  
4     matron_owner: ctx.accounts.buyer.key(), // should be matron cat's actual owner  
5     sire_owner: ctx.accounts.sire_owner.key(),  
6     attempt_id,  
7     sire_cat_id: sire_id,  
8     price: siring_fee,  
9 });
```

The impact of the given issue is:

Off-chain consumers — analytics dashboards, marketplace indexers, and revenue-tracking systems — that rely on `SiringUsed.matron_owner` to identify the matron's owner will receive the buyer's address instead. This produces incorrect player statistics and siring revenue attribution. The on-chain state itself is unaffected; the error is confined to the emitted event data.

Recommendation

Pass the matron cat's actual owner to the `SiringUsed` event. The matron `Cat` account is available in the instruction context; read `ctx.accounts.matron_cat.owner` and use it for the `matron_owner` field. Ensure the IDL description of this field is also updated to reflect the semantic intent.

Status

Acknowledged

Low 04 Signature Authority Rotation Immediately Invalidates All In-Flight Signed Messages

Location

config.rs:98-103

signature.rs:28-62

Description

`verify_ed25519_signature` reads `game_config.signature_authority` at execution time. When the admin calls `update_game_config` to rotate the SA key, the change takes effect immediately for every subsequent transaction. Any player who obtained a signed message from the old SA backend but has not yet submitted it on-chain will receive `GameError::InvalidSignature` on submission. There is no grace period, no dual-key acceptance window, and no on-chain signal warning players that their pending messages are about to expire.

```
1 // signature.rs:125 — SA key read from live config at verification time
2 if pubkey ≠ expected_signer.as_ref() {
3     continue;
4 }
```

The impact of the given issue is:

Players with pending join messages, breeding requests, or siring requests must re-fetch a new SA-signed message from the backend after any SA rotation. The user-visible symptom is an unexpected `InvalidSignature` error. No funds are at risk — unsubmitted messages carry no on-chain value. The disruption is amplified by the absence of an on-chain SA-rotation event (see I-11), meaning players learn about the rotation only by experiencing the error rather than from a proactive notification.

Recommendation

Consider implementing a two-key transition window: store both `signature_authority` (current) and `prev_signature_authority` (outgoing) in `GameConfig`, with a timestamp field indicating when the old key expires. During the window, accept messages signed by either key. This allows in-flight messages to drain normally before the old key is fully revoked. Separately, emit an on-chain Anchor event on SA rotation so monitoring systems and wallets can alert players in advance.

Status

Acknowledged

Low 05 `cancel_siring_listing` Executable After SOL And CAT Already Paid By Buyer

Location

`siring.rs`

Description

When `request_breeding_with_siring` executes, three irreversible state changes occur atomically: the buyer's SOL is transferred to sire owner and treasury, the buyer's CAT is burned, and the `BreedingAttempt` PDA is created with `STATUS_AWAITING_RANDOMNESS`. Critically, the sire's `cat.status` remains `STATUS_SIRING_LISTED` throughout - the handler never transitions it. `cancel_siring_listing` only checks `cat.status == STATUS_SIRING_LISTED`. No check exists that zero active `BreedingAttempt` PDAs reference this sire. So the sire owner can submit a cancel transaction in the same or next block after observing the buyer's paid transaction, close the listing PDA (recovering rent), and return the sire to IDLE - while the buyer's breeding proceeds normally. `resolve_breeding` does not check the sire's current status; it only reads the stored parent IDs from the `BreedingAttempt`. The child cat is created correctly, meaning the buyer suffers no direct financial loss. However the sire owner obtains listing rent on every use without maintaining the listing, and the sire returns to IDLE on cooldown — effectively monetizing the cancel/relist cycle as an optimization.

Proof of Concept

Let's have the following situation:

1. Sire owner lists cat at 0.005 SOL - listing PDA created, rent paid.

2. Buyer calls `request_breeding_with_siring`: 0.0045 SOL -> sire owner, 0.0005 SOL -> treasury, 500 CAT burned. `BreedingAttempt` created with `parent2_id = 1`.
3. Sire owner observes buyer TX on-chain, immediately calls `cancel_siring_listing`. Constraint: `cat.status == STATUS_SIRING_LISTED`. Listing PDA closed -> rent (~0.00175 SOL) returned to sire owner. `cat.status = STATUS_IDLE`.
4. VRF fulfills; operator calls `resolve_breeding`. Parent Cat PDAs for IDs in `BreedingAttempt` loaded - status not checked at resolve time. Child cat created correctly.
5. Sire owner profits from listing rent recovery on every paid siring, then relists after cooldown.

Recommendation

When `request_breeding_with_siring` executes, transition the sire status from `STATUS_SIRING_LISTED` to a locked state (e.g. reuse `STATUS_FARMING` conceptually, or add `STATUS_SIRING_IN_USE`) that blocks `cancel_siring_listing`. Restore to `STATUS_SIRING_LISTED` or `STATUS_IDLE` at `resolve_breeding` or `admin_close_attempt`.

Status

Acknowledged

Low 06 `join_fee` Not Committed In SA Signature — Admin Can Front-Run Join To Charge A Higher Fee

Location

`join.rs:132-141`

`signature.rs:207-237`

Description

The SA-signed join message commits to `player`, `cat_id_1`, `cat_id_2`, `generation`, `gp_1`, `gp_2`, `commitment_1`, `commitment_2`, `nonce`, and `expires_at` — but not to `join_fee`. The fee is read live from `game_config.join_fee` at execution time:

An admin can call `handle_update_game_config` to raise `join_fee` between the moment the player receives the SA-signed message and the moment the transaction lands. Because the signed message does not include the fee amount, the signature remains valid and the player is charged the new higher fee with no on-chain indication that it changed.

The `expires_at` field in the signed message provides a partial bound on the exposure window, but `expires_at` has no enforced upper bound (see I-03), so backends issuing long-lived messages widen the window proportionally.

The impact of the given issue is:

A player who accepted a fee of X lamports may be charged $Y > X$ lamports without their explicit consent. The discrepancy is bounded by however much the admin raises the fee — there is no cap on `join_fee` enforced on-chain. Since `join_fee` is denominated in SOL lamports, a large increase could cause the transaction to fail (if the player's wallet has insufficient balance) or silently drain more SOL than expected (if the wallet has enough). No funds are stolen from existing players; only the joining player is affected.

Recommendation

Include `join_fee` in the SA-signed message so the player's signature commits to the exact fee they agreed to pay:

```
1 // utils/signature.rs - build_join_message
2 format!(
3     "m:{}:jg:{}::{}:~{}:~{}:~{}:~{}:~{}:~{}:~{}:~{}:",
4     tag, player, cat_id_1, cat_id_2, generation,
5     gp_1, gp_2, commitment_1, commitment_2,
6     join_fee,    // <-- add this
7     nonce, expires_at
8 )
```

The handler then verifies the signed fee matches `game_config.join_fee` at execution time, rejecting the transaction if the fee changed after the message was issued.

Status

Acknowledged

Informational

Info 01 Immutable Placeholder CAT_TOKEN_URI Permanently Bricks Token Metadata Display On Deployment

Location

constants.rs

Description

CAT_TOKEN_URI = "https://arweave.net/hogehoge" is a placeholder URI. “Hogehoge” is the Japanese equivalent of “foobar” - a well-known dummy word in Japanese developer culture, not a valid Arweave transaction ID. A real Arweave TX ID is a 43-character base64url string. The URL at <https://arweave.net/hogehoge> does not return a 404 error. Instead, the arweave.net gateway directly resolves this path and serves a Nuxt.js HTML application (the Arweave Hub frontend) with a successful HTTP 200 status. This means wallets, explorers, and NFT indexers fetching this URI will receive a massive HTML document rather than the expected raw JSON metadata (containing `name`, `symbol`, `image`, `description`). The Token-2022 `initialize` metadata instruction writes this string permanently into the mint account at `initialize_system` time. There is no `update_token_metadata` instruction in this program. If deployed with this value, all token metadata display will fail parsing and be permanently broken with no on-chain recovery.

The `ar:// ...` reference in the comment is the ArNS protocol URI scheme (resolved natively by ArConnect and Arweave-aware browsers). The intended deployment flow per the comment is: upload image to Arweave -> upload `metadata.json` -> obtain the resulting 43-char TX ID -> set this constant to `https://arweave.net/<TX_ID>` -> rebuild and redeploy before calling `initialize_system`.

```
1 // === Token Metadata ===
2 pub const CAT_TOKEN_NAME: &str = "CAT";
3 pub const CAT_TOKEN_SYMBOL: &str = "CAT";
4 // TODO: Replace with real Arweave metadata URI before mainnet bootstrap.
5 // Upload flow (jailed-style script TBD): upload image -> upload metadata.json
6 // referencing the image -> set this constant to the resulting ar:// ... URL,
7 // then anchor build && redeploy && scripts/admin/launch.ts (Step 6).
```

```
8 pub const CAT_TOKEN_URI: &str = "https://arweave.net/hogehoge";
```

Recommendation

- **Primary Fix (Parameterize the URI):** Completely remove the hardcoded URI constant, as baking metadata links into the binary is an anti-pattern. Instead, require the URI as a standard string argument during the system initialization instruction, allowing the deployer to dynamically provide the real 43-character Arweave transaction ID.
- **Secondary Fix (Length-Based Compile-Time Guardrail):** If the team insists on keeping the URI hardcoded, implement a stricter compile-time assertion. Rather than just checking for the dummy word, assert that the string length meets the minimum character count of a valid Arweave URL (the gateway URL length plus the mandatory 43-character base64url ID).
- **Defense-in-Depth (Add an Update Path):** Implement a new admin-only instruction specifically to update the token metadata. This guarantees that if the decentralized storage link ever breaks or needs correction, the protocol authority can recover the token's visual state without having to deploy a completely new token mint.

Status

Acknowledged

Response: Deferred for later.

Info 02 Admin-Configurable Economic Parameters Lack On-Chain Bounds

Description

The three economic setter functions accept parameter values across their full mathematical range without on-chain guardrails that would prevent obviously degenerate configurations. `breeding_cost` accepts zero, eliminating the primary CAT burn mechanism. `marketplace_fee_rate` accepts both zero (no treasury revenue from sales) and 10,000 basis points (sellers and sire owners receive nothing). `min_sale_price` and `min_siring_price` accept zero, enabling dust-price spam listings. No multi-step confirmation, time-lock, or hard floor prevents any of these configurations from taking effect immediately in a single transaction.

```
1 // admin/config.rs:104-110 - no floor check for breeding_cost or max_fancy_gp
2 if let Some(v) = max_fancy_gp {
3     config.max_fancy_gp = v;    // accepts 0
4 }
5 if let Some(v) = breeding_cost {
6     config.breeding_cost = v;    // accepts 0
7 }
8
9 // admin/config.rs:149-159 - only upper bound enforced for fee rate
10 require!(
11     v ≤ u16::try_from(crate::constants::FEE_DENOMINATOR).expect(" ... "),
12     crate::errors::GameError::InvalidParameter
13 );
```

The impact of the given issue is:

Under deliberate admin action: setting `breeding_cost = 0` allows unlimited free breeding, inflating CAT supply without the deflationary burn that the tokenomics model depends on; setting `marketplace_fee_rate = 0` eliminates all protocol marketplace revenue; setting `marketplace_fee_rate = 10000` extracts 100% of every sale price from sellers and sire owners; setting `min_sale_price = 0` enables zero-cost cat transfers and marketplace spam. All scenarios require the admin to act contrary to the protocol's documented economic intent.

Recommendation

Define protocol-minimum and protocol-maximum constants for each economic parameter and enforce them on-chain:

- `breeding_cost`: minimum floor of, for example, `MIN_BREEDING_COST = 1` (or a meaningful economic floor agreed with tokenomics design).
- `marketplace_fee_rate`: minimum floor of 1 bps and maximum operational cap of, for example, `2_500` bps (25%), separate from the mathematical maximum of 10,000 bps.
- `min_sale_price` / `min_siring_price`: minimum floor of at least 1 lamport (or a dust-prevention minimum agreed with the marketplace design).

Status

Acknowledged

Response: Admin economic bounds are intentional.

Info 03 No On-Chain Anchor Events for 12 Admin Functions

Location

config.rs

pause.rs

emergency.rs

Description

Every admin state-changing function in the program logs via `msg!()` but emits no structured Anchor `emit!()` event. An audit of all admin handlers confirms the pattern:

Function	File	<code>emit!()</code> ?
<code>handle_update_emission_table</code>	<code>config.rs:35</code>	No
<code>handle_update_game_config</code>	<code>config.rs:72</code>	No
<code>handle_update_marketplace_config</code>	<code>config.rs:134</code>	No
<code>handle_add_operator</code>	<code>operator.rs:24</code>	No
<code>handle_remove_operator</code>	<code>operator.rs:67</code>	No
<code>handle_toggle_function_pause</code>	<code>pause.rs:58</code>	No
<code>handle_toggle_global_pause</code>	<code>pause.rs:91</code>	No
<code>handle_admin_create_cat</code>	<code>emergency.rs:46</code>	No
<code>handle_admin_close_attempt</code>	<code>emergency.rs:115</code>	No
<code>handle_reset_player_nonce</code>	<code>emergency.rs:198</code>	No
<code>handle_bootstrap_mint</code>	<code>emergency.rs:268</code>	No
<code>handle_initialize_system</code>	<code>initialize.rs:56</code>	No

By contrast, all player-facing instructions (`join_game`, `batch_update_farm`, `claim_rewards`, `request_breeding`, `resolve_breeding`, `marketplace` instructions) correctly use `emit!()`.

Anchor `emit!()` events are encoded in a standardized format that is parsed and indexed by Anchor client libraries, Helius webhooks, and standard RPC program-subscription endpoints. `msg!()` output is written to transaction logs, which are ephemeral, not easily filterable without full-log-indexing infrastructure, and not subscribable via standard `programSubscribe` event channels.

The impact of the given issue is:

- Monitoring systems relying on Anchor event subscriptions receive no signal for any admin state change.
- Detection latency for critical security-relevant changes (admin key rotation, SA rotation, `max_fancy_gp = 0`, nonce reset) is bounded by the operator's `GameConfig` polling cadence, not by real-time event subscription.
- Incident response is impaired: when a config change causes unexpected behavior, operators cannot reconstruct the change timeline from event history alone.

Recommendation

Define Anchor `Event` structs for each admin operation and replace `msg!()` with `emit!()` at every admin state-change site. At minimum, prioritize the five highest-security-impact events:

```

1  #[event]
2  pub struct GameConfigUpdated {
3      pub admin: Pubkey,
4      pub signature_authority_changed: bool,
5      pub max_fancy_gp: Option<u32>,
6      pub breeding_cost: Option<u64>,
7      pub timestamp: i64,
8  }
9
10 #[event]
11 pub struct PlayerNonceReset {
12     pub admin: Pubkey,
13     pub player: Pubkey,
14     pub old_nonce: u64,
15     pub timestamp: i64,
16 }
17
18 #[event]
19 pub struct PauseStateChanged {
20     pub admin: Pubkey,
21     pub global_pause: bool,
22     pub paused_functions: u64,
23     pub timestamp: i64,
24 }
```

Status

Fixed

Info 04 cancel_sale_listing And cancel_siring_listing Missing Explicit `cat.owner = seller` Constraint

Location

sale.rs

siring.rs

Description

Both cancel handlers verify `listing.seller = seller.key()` and validate the listing PDA address, but neither adds `constraint = cat.owner = seller.key()`. Currently unexploitable because no instruction can modify `Cat.owner` while the cat has `STATUS_SALE_LISTED` or `STATUS_SIRING_LISTED`. However, the ownership assertion is missing from the struct level, violating the defense-in-depth principle the rest of the program follows. Any future instruction that writes `cat.owner` without touching status would make cancel exploitable without any change to the cancel handlers.

Recommendation

Add `constraint = cat.owner = seller.key() @ GameError::NotCatOwner` to both cancel account structs.

Status

Fixed

Info 05 placed_cat_ids Array Uses 0 As Empty Sentinel — Collides With Valid Cat ID 0

Location

state.rs:92-103

state.rs:86-89

Description

`PlayerState.placed_cat_ids` is a fixed-length `[u32; MAX_PLACED_CATS]` array. The swap-remove in `remove_placed_cat` clears the vacated tail slot by writing 0:

```

1 // state.rs:92-103
2 pub fn remove_placed_cat(&mut self, cat_id: u32) -> bool {
3     let count = self.placed_count as usize;
4     for i in 0..count {
5         if self.placed_cat_ids[i] == cat_id {
6             self.placed_cat_ids[i] = self.placed_cat_ids[count - 1];
7             self.placed_cat_ids[count - 1] = 0; // <-- sentinel = 0
8             self.placed_count -= 1;
9             return true;
10        }
11    }
12    false
13 }
```

0 is the sentinel for “empty slot”. However, `cat_id` is a `u32` and the protocol imposes no on-chain lower-bound guard — nothing prevents a cat with `id = 0` from being created (e.g., via `admin_create_cat` if the SA were to issue ID 0, or if a future instruction path produces an attempt whose derived `cat_id` resolves to 0).

If cat ID 0 were placed in the farm, two bugs occur simultaneously:

1. False removal: any call to `remove_placed_cat(0)` would find the first occurrence of 0 in the array. But after the swap-remove cleanup, the tail slot is also written to 0 — if cat ID 0 was at a non-tail position the swap brings the last real cat ID to position `i`, then writes 0 at tail and decrements `placed_count`. If the tail slot already contained ID 0 (either as the cat itself or as a stale empty sentinel), the removal looks correct but `placed_count` could be decremented for a sentinel rather than a real entry.
2. Phantom removal: if the array contains `[0, X, Y, 0]` with `placed_count = 3` (0 is a real cat, X and Y are cats, last slot is sentinel), `remove_placed_cat(0)` would match index 0, swap X into position 0, write 0 at index 2, decrement count to 2 — losing Y silently.

The current SA-driven join flow assigns cat IDs that are operationally nonzero (the genesis bootstrap starts from ID 1 by convention), so the bug is not exploitable today. There is, however, no `require!(cat_id != 0)` guard in `join_game`, `add_placed_cat`, or `admin_create_cat` to make this invariant explicit and permanent.

The impact of the given issue is:

No funds are at risk under current operational assumptions — the SA never issues cat ID 0. The risk is latent: any future path (admin tool error, instruction reuse, off-chain numbering change) that creates a cat with ID 0 would silently corrupt the `placed_cat_ids` array, causing a placed cat

to disappear from tracking without the standard `batch_update_farm` removal flow. A disappeared cat would retain its `STATUS_FARMING` on-chain status (the `Cat` account is unchanged) but would no longer be counted in `placed_count`, creating a permanent discrepancy between the farm array and the cat's on-chain status.

Recommendation

Add an explicit on-chain guard that cat ID 0 is never issued or placed:

```
1 // In join_game (or add_placed_cat):
2 require!(cat_id_1 != 0 && cat_id_2 != 0, GameError::InvalidCatId);
3
4 // In admin_create_cat:
5 require!(cat_id != 0, GameError::InvalidCatId);
```

Alternatively, replace the sentinel value with `u32::MAX` (which the `cat_id` counter will never reach in practice) to eliminate the ambiguity entirely. This requires only changing the tail-clear line in `remove_placed_cat` and updating any initialization that zero-fills the array.

Status

Fixed